

Design-Quality Review

1. Anomaly and Redundancy Analysis

The first redundancy in the schema is the deliberate duplication of the `requires_approval` flag. It appears both in the `room` table (as the room's current policy) and in the `booking` table (as a snapshot taken at booking creation time).

This is a **justified denormalization**, but it still introduces an **update anomaly risk**: if a room's approval policy changes, the `room.requires_approval` value is updated, but all historical `booking.requires_approval` snapshots remain unchanged. This is actually the *intended* behavior: historical bookings must not be retroactively affected. Future developers should not treat both columns as equivalent and attempt to synchronize them, corrupting the audit trail.

A secondary risk exists in the `booking_series` table. It stores `room_id`, `organization_id`, and `created_by`, and each individual `booking` occurrence also stores these same three foreign keys independently. If a booking is later associated with a different organization (e.g., a correction), the series record and its occurrence records can fall out of sync. There is no schema-level constraint enforcing that `booking.organization_id` matches `booking_series.organization_id` for bookings that belong to that series. This is an **insertion/update anomaly risk** in the current design.

2. Functional Dependencies

FD 1: `booking_id` → `requires_approval` (on the `booking` table)

A booking's `requires_approval` value is determined entirely by its `booking_id`: it is a snapshot of the room's approval policy taken at the moment the booking was created.

This FD is significant because it sits in tension with the analogous FD `room_id` → `requires_approval` on the `room` table. Both columns carry the same semantic label but serve different purposes. The booking-level snapshot breaks the normal expectation that approval policy is a property of the room alone. This is the root of the update anomaly described above: if `room.requires_approval` is updated, there is no cascade to historical bookings, which is correct by design but must be enforced by application logic rather than the schema.

FD 2: (`room_id`, `equipment_id`) → `quantity` (on the `room_equipment` table)

The quantity of a given equipment type in a given room is fully determined by the combination of room and equipment type.

This FD confirms that `room_equipment` is in **Boyce-Codd Normal Form (BCNF)**: the composite primary key (`room_id`, `equipment_id`) is the only determinant, and `quantity` depends on the whole key, not a subset. No anomaly exists here. The FD is noted because it justifies keeping `quantity` in the linking table rather than on `equipment_type`, and moving it there would incorrectly imply that quantity is a property of the equipment type globally, rather than of a specific room's inventory.

FD 3: `series_id` → (`room_id`, `organization_id`, `created_by`, `day_of_week`, `start_time`, `end_time`) (on `booking_series`)

All descriptive attributes of a recurring booking series are determined by the series identifier alone.

This FD is clean and well-structured. However, it creates an implicit *expected* FD between `booking_series` and `booking`: for any booking where `series_id` IS NOT NULL, one would expect `booking.room_id = booking_series.room_id` and `booking.organization_id = booking_series.organization_id`. This expectation is not enforced by any CHECK constraint in the current schema, which is the anomaly risk flagged in Section 1. A booking occurrence could, in theory, be inserted with a mismatched `room_id` relative to its parent series, and the schema would accept it silently.

3. Schema Revisions

The schema was reviewed against the three functional dependencies above and the anomaly risks identified. Two decisions were made:

No structural change to `requires_approval` duplication. The snapshot pattern is retained as-is. The risk is real but the design intent is correct: `room.requires_approval` reflects current policy; `booking.requires_approval` is an immutable historical record. The CHECK constraint on `booking` already enforces consistency between `requires_approval`, `approval_granted`, and `status` at the row level, which is the appropriate place for this logic. Adding a trigger to copy `room.requires_approval` into `booking.requires_approval` at INSERT time would be a reasonable application-layer safeguard, but is outside the scope of the schema definition itself.

A note is added regarding series-occurrence consistency. The schema does not currently enforce that `booking.room_id` matches `booking_series.room_id` when a `series_id` is present. In a production system, this would be addressed with either a trigger or an application-layer validation. For the scope of this project, the risk is documented as a known limitation rather than resolved structurally, since adding a cross-table CHECK

constraint enforcing this relationship is not supported natively in PostgreSQL without a trigger.

The **booking_series.organization_id** column is correctly nullable (to match `booking.organization_id`, which is also nullable: a person may book without an organization). This was verified to be consistent across both tables in the current schema.

4. Privacy Review

The `person` table stores only a name and a unique email address, the minimum needed to identify who created a booking and to enable contact if a booking needs to be managed or cancelled. No phone numbers, student IDs, roles, or affiliations are stored, which is consistent with the case requirement to keep personal data to a minimum. The `cancellation_reason` field on `booking` is the most significant privacy risk in the schema: it is a free-text field with no length constraint or content validation, and a user could enter personal or sensitive information (for example, a medical reason) that the system has no mechanism to detect or restrict. This field should be treated as potentially sensitive personal data in any access control policy, and operators should be advised to keep entries brief and non-identifying. Finally, because `person_id` is referenced by both `booking` and `booking_series`, deleting a person record is blocked by foreign key constraints. The system should define a clear data retention and deletion policy to handle right-to-erasure requests without orphaning booking history.